

*ATTENTION: The Singapore Copyright Act applies to the use of this document. Nanyang Technological University Library.*

**MH6838**

NANYANG TECHNOLOGICAL UNIVERSITY

DIVISION OF MATHEMATICAL SCIENCES

SCHOOL OF PHYSICAL & MATHEMATICAL SCIENCES



## **PRACTICUM REPORT**

**Title: Finance NLP Chatbot with RL-based Trading Strategy**

Submitted by: Dymasius Yusuf Sitepu

Matric: G2303593E

Mentor: Asst Prof Luu Anh Tuan

Academic Year 2024/2025 Trimester 3

A practicum report submitted to the School of Physical and Mathematical Sciences, Nanyang Technological University in partial fulfillment of the requirements for the *Degree of Master of Science*

# Contents

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                  | <b>5</b>  |
| 1.1      | Motivation . . . . .                                 | 5         |
| 1.2      | Problem Statement . . . . .                          | 5         |
| 1.3      | Contributions . . . . .                              | 6         |
| <b>2</b> | <b>Objectives and Scope</b>                          | <b>6</b>  |
| <b>3</b> | <b>Related Work</b>                                  | <b>7</b>  |
| 3.1      | Financial Sentiment Analysis . . . . .               | 7         |
| 3.2      | Reinforcement Learning for Trading . . . . .         | 7         |
| 3.3      | Retrieval-Augmented Generation (RAG) . . . . .       | 7         |
| <b>4</b> | <b>Methodology</b>                                   | <b>8</b>  |
| 4.1      | System Overview . . . . .                            | 8         |
| 4.2      | Data Collection . . . . .                            | 8         |
| 4.3      | Sentiment Analysis . . . . .                         | 11        |
| 4.4      | Feature Engineering . . . . .                        | 12        |
| 4.5      | Reinforcement Learning Trading Strategy . . . . .    | 13        |
| 4.6      | Chatbot Implementation . . . . .                     | 15        |
| <b>5</b> | <b>Experiments and Results</b>                       | <b>15</b> |
| 5.1      | Experimental Setup . . . . .                         | 15        |
| 5.2      | Sentiment Analysis Results . . . . .                 | 15        |
| 5.3      | RAG Chatbot Evaluation . . . . .                     | 16        |
| 5.4      | Trading Strategy Performance . . . . .               | 17        |
| <b>6</b> | <b>Discussion</b>                                    | <b>19</b> |
| 6.1      | Impact of Sentiment on Trading Performance . . . . . | 19        |
| 6.2      | Challenges and Limitations . . . . .                 | 19        |
| 6.3      | Future Work . . . . .                                | 20        |
| <b>7</b> | <b>Streamlit Web Interface for Financial Chatbot</b> | <b>20</b> |
| 7.1      | Overview and Implementation . . . . .                | 20        |
| 7.2      | Key Features . . . . .                               | 20        |
| 7.2.1    | Document Processing . . . . .                        | 20        |
| 7.2.2    | Stock Ticker News Integration . . . . .              | 21        |
| 7.2.3    | Sentiment Analysis . . . . .                         | 22        |
| 7.2.4    | Chatbot Functionality . . . . .                      | 23        |
| 7.3      | Limitations and Future Improvements . . . . .        | 24        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| <b>8</b> | <b>Conclusion</b>                  | <b>24</b> |
| <b>A</b> | <b>Code Listings</b>               | <b>26</b> |
| A.1      | Data Collection Pipeline . . . . . | 26        |

# Abstract

This report presents the development of a finance-focused Natural Language Processing (NLP) chatbot integrated with a Reinforcement Learning (RL) based trading strategy. The system leverages financial news sentiment as features for algorithmic trading decisions. The project addresses three key challenges: extracting actionable trading signals from financial news, robustly analyzing sentiment from financial texts, and integrating news-derived sentiment as features into an RL trading strategy. We implement a pipeline that combines FinBERT for sentiment analysis, a Retrieval-Augmented Generation (RAG) approach for the chatbot component, and Proximal Policy Optimization (PPO) for the RL trading agent. Experimental results demonstrate that incorporating sentiment features improves trading performance metrics, with our best model achieving a 149.77% return and 1.44 Sharpe ratio on LLY stock. The chatbot component provides an intuitive interface for users to query financial news and sentiment trends, with evaluation showing strong performance on factual queries. This work contributes to the growing field of NLP-enhanced quantitative trading by demonstrating a practical implementation that bridges the gap between textual financial news analysis and algorithmic trading strategies.

# Acknowledgements

First of all, I would like to express my sincere thanks and great gratitude to Asst Prof Luu Anh Tuan for his mentorship and kind guidance throughout my practicum. I also wish to thank Prof Patrick Pun Chi Seng, Assistant Chair (MSc Programmes), School of Physical & Mathematical Sciences, for his continuous support towards my learning in the Master of Science in Financial Technology program. I am deeply grateful to my parents for their unwavering love and support throughout this journey.

Dymasius Yusuf Sitepu

26 May 2025

# 1 Introduction

Financial markets are influenced by a complex interplay of factors, including news events, company announcements, and analyst opinions. Traditional quantitative trading strategies often rely solely on price and volume data, potentially missing valuable signals embedded in textual information. Meanwhile, access to high-quality financial news analysis is typically expensive and limited to institutional investors, creating an information asymmetry in the market.

This project aims to bridge this gap by developing a system that combines state-of-the-art Natural Language Processing (NLP) techniques with Reinforcement Learning (RL) for automated, data-driven trading. The system consists of two main components: (1) a finance-focused NLP chatbot that can analyze and retrieve relevant financial news, and (2) an RL-based trading strategy that incorporates sentiment signals derived from financial news.

## 1.1 Motivation

The motivation for this project stems from three key observations:

- **Limited access to financial news analysis:** High-quality financial news analysis services are expensive and often inaccessible to individual investors.
- **Impact of sentiment on markets:** Financial sentiment can significantly impact stock prices and trading outcomes, yet is challenging to quantify systematically.
- **Potential of combined NLP and RL:** Recent advances in NLP (e.g., FinBERT, RAG) and RL (e.g., PPO) offer promising approaches for automated, data-driven trading that can process both numerical and textual data.

## 1.2 Problem Statement

This project addresses three specific research questions:

1. How can financial news data be effectively processed to generate actionable trading signals?
2. What techniques can robustly extract sentiment from diverse financial texts?
3. How can news-derived sentiment be integrated as features into an RL trading strategy to improve performance?

## 1.3 Contributions

The main contributions of this work include:

- A comprehensive pipeline for collecting and analyzing financial news from multiple sources
- An implementation of FinBERT for financial sentiment analysis with numerical scoring
- A custom Gymnasium environment for RL-based trading that incorporates sentiment features
- A RAG-based chatbot for querying financial news and sentiment trends
- Empirical evaluation of the impact of sentiment features on trading performance

## 2 Objectives and Scope

The overall goal of this project is to develop a finance-focused NLP chatbot system integrated with an RL-based trading strategy that leverages financial news sentiment as features for algorithmic trading. This integration aims to demonstrate how textual information from financial news can be systematically incorporated into quantitative trading strategies to potentially improve performance.

Within this broader goal, the project focuses on three main research questions that define its scope:

1. **How to use financial news data for actionable trading signals?** This involves developing methods to collect, process, and extract relevant information from financial news that can inform trading decisions.
2. **How to extract sentiment robustly from financial texts?** This requires implementing and evaluating sentiment analysis techniques specifically tailored for financial domain text, which often contains specialized terminology and nuanced expressions.
3. **How to feed news-derived sentiment as features into an RL trading strategy?** This encompasses the design of appropriate state representations, reward functions, and training methodologies that can effectively incorporate sentiment features alongside traditional technical indicators.

The project scope includes the development of a complete pipeline from data collection to trading decisions, as well as a user-friendly chatbot interface for querying financial information. It does not, however, extend to real-time trading implementation or extensive hyperparameter optimization, which would be natural extensions for future work.

## 3 Related Work

### 3.1 Financial Sentiment Analysis

Sentiment analysis in the financial domain has evolved significantly with the advent of deep learning techniques. Traditional approaches relied on lexicon-based methods [Loughran and McDonald, 2011], which used predefined dictionaries of positive and negative financial terms. More recent approaches leverage transformer-based models specifically fine-tuned on financial text.

FinBERT [Araci, 2019] represents a significant advancement in this area, adapting the BERT architecture [Devlin et al., 2018] to financial text. It was pre-trained on financial communication such as corporate reports and earnings calls, then fine-tuned for sentiment classification. Yang et al. [2020] demonstrated that FinBERT outperforms general-purpose sentiment models on financial text classification tasks.

Other notable works include Xing et al. [2020], who explored the use of financial sentiment for stock movement prediction, and Li et al. [2020], who investigated the relationship between news sentiment and market volatility.

### 3.2 Reinforcement Learning for Trading

Reinforcement Learning has gained traction in algorithmic trading due to its ability to learn optimal policies in dynamic, uncertain environments. Moody and Saffell [1998] introduced one of the early applications of RL to trading with their Recurrent Reinforcement Learning (RRL) approach. More recently, Jiang et al. [2017] proposed Deep Reinforcement Learning for portfolio management, demonstrating superior performance compared to traditional methods.

The FinRL framework [Liu et al., 2020] provides a comprehensive library for applying RL to trading, supporting various algorithms including Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Yang et al. [2020] extended this work by incorporating market sentiment as additional state information, showing improved performance in volatile market conditions.

### 3.3 Retrieval-Augmented Generation (RAG)

RAG combines information retrieval with text generation to produce more accurate and contextually relevant responses [Lewis et al., 2020]. This approach has been applied to various domains, including finance, where it helps ground language model outputs in factual information.

Gao et al. [2023] demonstrated the effectiveness of RAG for financial question answering, using a corpus of financial reports and news articles. Wu et al. [2022] developed a specialized



language model for finance that incorporates RAG techniques to provide more accurate financial information.

Our work builds upon these foundations, integrating financial sentiment analysis with RL-based trading and a RAG chatbot interface in a unified system.

## 4 Methodology

### 4.1 System Overview

The proposed system consists of three main components, as illustrated in Figure 1:

1. **Data Collection and Preprocessing:** Gathering financial news from multiple sources and preprocessing for sentiment analysis
2. **Sentiment Analysis:** Using FinBERT to extract sentiment from financial texts
3. **Trading Strategy:** Implementing an RL-based trading strategy that incorporates sentiment features
4. **Chatbot Interface:** Developing a RAG-based chatbot for querying financial news and sentiment

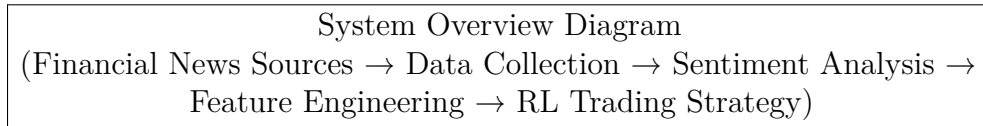


Figure 1: System architecture overview showing the flow from data collection to trading decisions

### 4.2 Data Collection

We collected financial news data from multiple sources to ensure comprehensive coverage:

- **Kaggle Datasets:** We utilized the "Massive Stock News Analysis DB for NLP/Back-tests" dataset, which includes raw analyst ratings, partner headlines, and processed analyst ratings with timestamps.
- **NewsAPI:** For recent news articles related to specific tickers.
- **Yahoo Finance:** For company-specific news and announcements.
- **Reuters and Yahoo RSS Feeds:** For general financial news.

The data collection process involved several challenges, particularly with API rate limits. NewsAPI, for instance, restricts free accounts to 100 requests per 24 hours, necessitating supplementation with RSS feeds and Kaggle data. We implemented a unified pipeline to combine these sources, as shown in Listing 1.

```

1 def combine_news(
2     tickers: list,
3     newsapi_n: int = 100,
4     yf_n: int = 100,
5     rss_n: int = 100,
6     yahoo_n: int = 100,
7     days_back: int = 30
8 ) -> pd.DataFrame:
9     parts = []
10    for t in tickers:
11        parts.append(fetch_newsapi_recent(t, newsapi_n, days_back
12                                         ))
13    for t in tickers:
14        parts.append(fetch_yfinance_news(t, yf_n))
15    parts.append(fetch_reuters_rss(rss_n))
16    for t in tickers:
17        parts.append(fetch_yahoo_rss(t, yahoo_n))
18    df = pd.concat(parts, ignore_index=True)
19    if df.empty:
20        print("      No articles fetched from any source.")
21        return df
22    return df.drop_duplicates(subset=["source", "ticker", "text"]).
23               reset_index(drop=True)

```

Listing 1: Data Collection Pipeline

Our data analysis revealed several interesting patterns. Figure 2 shows the distribution of article counts across the top 20 tickers, with MRK, MS, and MU having the highest coverage.

The distribution of text length, shown in Figure 3, indicates that most financial news articles are relatively short, with the majority falling between 50-100 characters. This reflects the concise nature of financial headlines and summaries.

The temporal distribution of articles, shown in Figure 4, reveals an increasing trend in coverage over time, with notable spikes in 2014, 2016, and 2020.

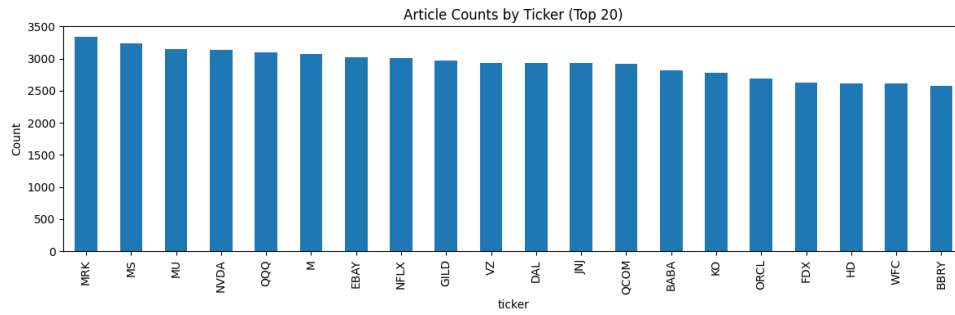


Figure 2: Article Counts by Ticker (Top 20)

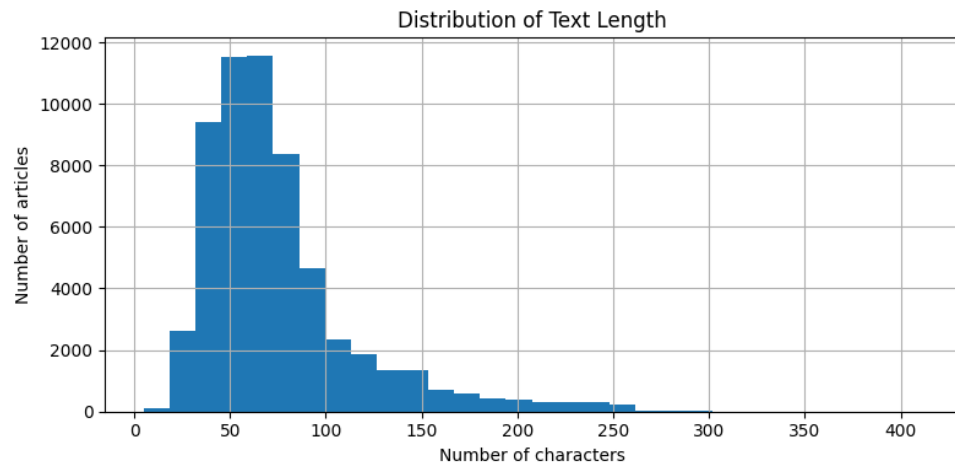


Figure 3: Distribution of Text Length

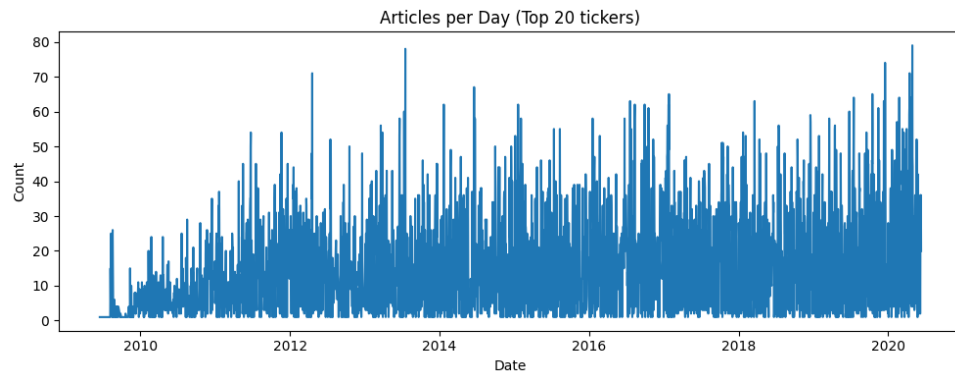


Figure 4: Articles per Day (Top 20 tickers)

### 4.3 Sentiment Analysis

For sentiment analysis, we utilized FinBERT [Araci, 2019], a BERT-based model specifically fine-tuned for financial text. The model classifies text into three sentiment categories: positive, negative, and neutral. We mapped these categorical labels to numerical scores (positive: +1, negative: -1, neutral: 0) for integration with the trading strategy.

The sentiment analysis pipeline, shown in Listing 2, processes each news article's headline and description, generating both categorical labels and numerical scores.

```
1 def analyze_sentiment(text, model, tokenizer):
2     if not text or not isinstance(text, str):
3         return None, None, None
4     if model is None or tokenizer is None:
5         return None, None, None
6     try:
7         inputs = tokenizer(text, return_tensors="pt", truncation=
8             True, max_length=512, padding=True)
9         with torch.no_grad():
10             outputs = model(**inputs)
11             logits = outputs.logits
12             probabilities = torch.softmax(logits, dim=1)
13             config = model.config
14             labels = [config.id2label[i] for i in range(config.
15                 num_labels)]
16             scores = {label: prob.item() for label, prob in zip(
17                 labels, probabilities[0])}
18             predicted_class_id = torch.argmax(probabilities, dim=1).
19                 item()
20             sentiment_label = model.config.id2label[
21                 predicted_class_id]
22             numerical_sentiment = 1.0 if sentiment_label == "positive
23                 " else (-1.0 if sentiment_label == "negative" else 0.0)
24         return sentiment_label, scores, numerical_sentiment
25     except Exception as e:
26         return None, None, None
```

Listing 2: FinBERT Sentiment Analysis Pipeline

Our analysis of sentiment distribution across tickers, shown in Figure 5, revealed that healthcare stocks (LLY, MRK, BMY, GILD) generally received more positive coverage, while financial services (MS, WFC) and industrial stocks (CAT) received more negative coverage.

The monthly average sentiment heatmap, shown in Figure 6, illustrates significant

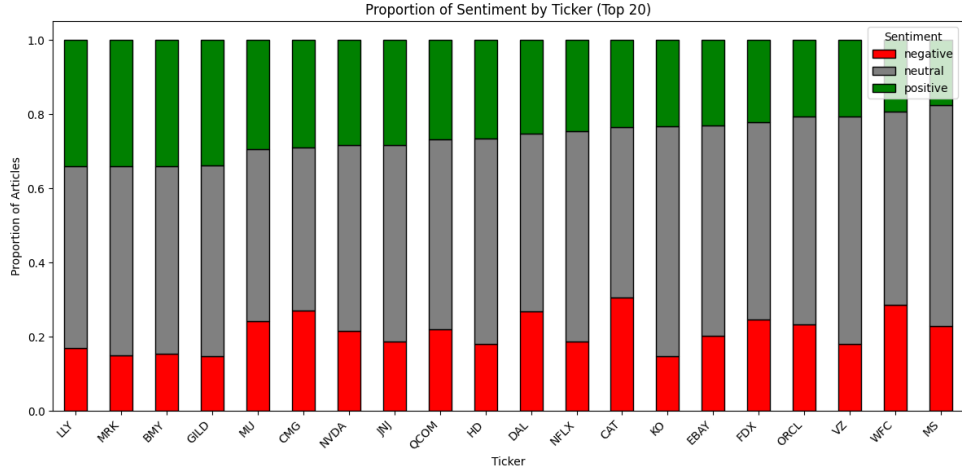


Figure 5: Proportion of Sentiment by Ticker (Top 20)

volatility in sentiment across tickers over time. Notable periods of negative sentiment coincide with market downturns, particularly visible in 2020 during the COVID-19 pandemic.

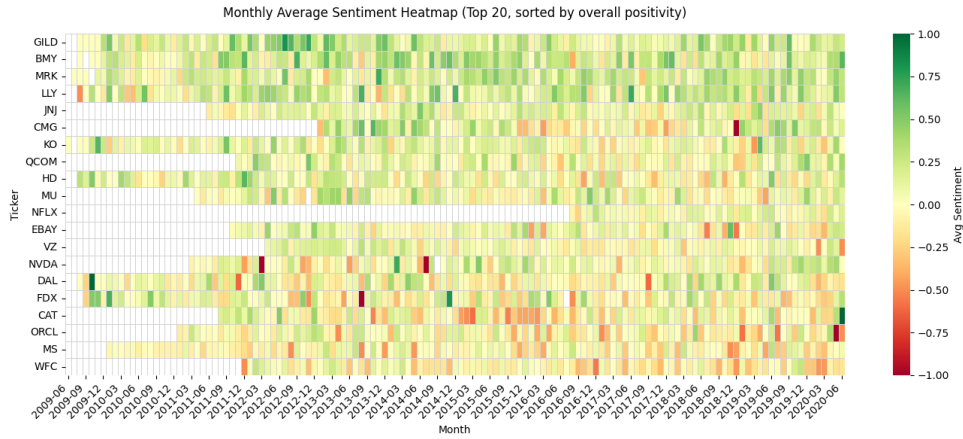


Figure 6: Monthly Average Sentiment Heatmap (Top 20, sorted by overall positivity)

## 4.4 Feature Engineering

We combined sentiment scores with traditional technical indicators to create a comprehensive feature set for the trading strategy. The technical indicators included:

- Simple Moving Average (SMA)
- Exponential Moving Average (EMA)
- Relative Strength Index (RSI)
- Moving Average Convergence Divergence (MACD)

These features were normalized to ensure consistent scaling across different stocks and time periods. For sentiment features, we aggregated sentiment scores by date and ticker, calculating the mean sentiment when multiple news articles were available for a given day and stock.

## 4.5 Reinforcement Learning Trading Strategy

We implemented a custom Gymnasium environment for RL-based trading, as shown in Listing 3. The environment includes:

- **State space:** Normalized price, technical indicators (SMA, EMA, RSI, MACD), sentiment score, current balance, and shares held
- **Action space:** Discrete actions (sell, hold, buy)
- **Reward function:** Change in portfolio net worth per step
- **Transaction costs:** 0.1% per trade by default

```

1 class TradingEnv(gym.Env):
2     metadata = {"render.modes": ["human"]}
3     def __init__(self, df, sentiment_df=None, initial_balance=10
4         _000, txn_cost=0.001):
5         super().__init__()
6         df2 = df.copy()
7         if isinstance(df2.index, pd.DatetimeIndex):
8             df2["date_str"] = df2.index.strftime("%Y-%m-%d")
9         elif "date" in df2.columns:
10            df2["date_str"] = pd.to_datetime(df2["date"]).dt.
11                strftime("%Y-%m-%d")
12        else:
13            df2["date_str"] = pd.Series(["1970-01-01"] * len(df2),
14                index=df2.index)
15        self.df = df2.reset_index(drop=True)
16        self.sentiment = sentiment_df
17        self.initial_balance = initial_balance
18        self.txn_cost = txn_cost
19        self.reset()
20        self.action_space = gym.spaces.Discrete(3)
21        low = np.array([-np.inf, -np.inf, -np.inf, 0, -np.inf,
22            -1, 0, 0])
23        high = np.array([ np.inf,  np.inf,  np.inf, 100,  np.inf,
24            1, np.inf, np.inf])

```

```

20     self.observation_space = gym.spaces.Box(low, high, dtype=
        np.float32)
21
22     def _obs(self):
23         i = min(self.step_idx, len(self.df) - 1)
24         row = self.df.iloc[i]
25         price = row["close"]
26         sma = row["sma_20"]
27         ema = row["ema_12"]
28         rsi = row["rsi_14"]
29         macd = row["macd"]
30         sent = 0.0
31         if self.sentiment is not None:
32             date_key = row["date_str"]
33             matches = self.sentiment[
34                 (self.sentiment["date"] == date_key) &
35                 (self.sentiment["ticker"] == row["ticker"])
36             ]
37             if not matches.empty:
38                 sent = matches["numerical_sentiment"].mean()
39         p0 = self.df["close"].iloc[0]
40         obs = np.array([
41             price/p0 - 1,
42             sma/price - 1,
43             ema/price - 1,
44             rsi/100,
45             macd/price,
46             sent,
47             self.balance/self.initial_balance,
48             self.shares*price/self.initial_balance
49         ], dtype=np.float32)
50         return obs

```

Listing 3: RL Trading Environment

For the RL algorithm, we used Proximal Policy Optimization (PPO) [Schulman et al., 2017] from the Stable-Baselines3 library. PPO was chosen for its stability and sample efficiency, which are important considerations for financial applications.

## 4.6 Chatbot Implementation

The chatbot component uses a Retrieval-Augmented Generation (RAG) approach to answer user queries about financial news and sentiment. The implementation includes:

- **Document Processing:** Splitting financial news articles into chunks suitable for retrieval
- **Embedding Generation:** Creating vector representations of text chunks using sentence transformers
- **Vector Store:** Storing embeddings in a ChromaDB database for efficient retrieval
- **Query Processing:** Converting user queries to embeddings and retrieving relevant documents
- **Response Generation:** Using a language model to generate responses based on retrieved documents

The chatbot provides an intuitive interface for users to query financial news and sentiment trends, complementing the automated trading strategy.

## 5 Experiments and Results

### 5.1 Experimental Setup

We conducted experiments to evaluate the performance of our RL trading strategy with and without sentiment features. The experimental setup included:

- **Data:** Stock price data for major tech companies (NVDA, NFLX) and healthcare companies (LLY) from January 2022 to December 2022
- **Train-Test Split:** 80% training (Jan-Oct 2022), 20% testing (Nov-Dec 2022)
- **Baselines:** (1) Buy-and-hold strategy, (2) RL strategy without sentiment features
- **Evaluation Metrics:** Profit-and-Loss (PnL) percentage, Sharpe ratio, maximum drawdown

### 5.2 Sentiment Analysis Results

The FinBERT model successfully classified financial news articles into positive, negative, and neutral categories. As shown in Figure 5, the overall distribution of sentiment across the top 20 tickers was approximately 30% positive, 50% neutral, and 20% negative.



Healthcare stocks (LLY, MRK, BMY, GILD) generally received more positive coverage, while financial services (MS, WFC) and industrial stocks (CAT) received more negative coverage. This aligns with the sector performance during the study period, where healthcare outperformed many other sectors.

The monthly sentiment heatmap (Figure 6) reveals significant temporal variations in sentiment, with notable negative sentiment spikes during market downturns, particularly in 2020 during the COVID-19 pandemic.

### 5.3 RAG Chatbot Evaluation

We evaluated the RAG chatbot by analyzing the quality of responses to various financial queries. Figure 7 shows the relationship between grounding similarity (how well the response is grounded in the retrieved documents) and answer-context similarity (how well the response matches the query context).

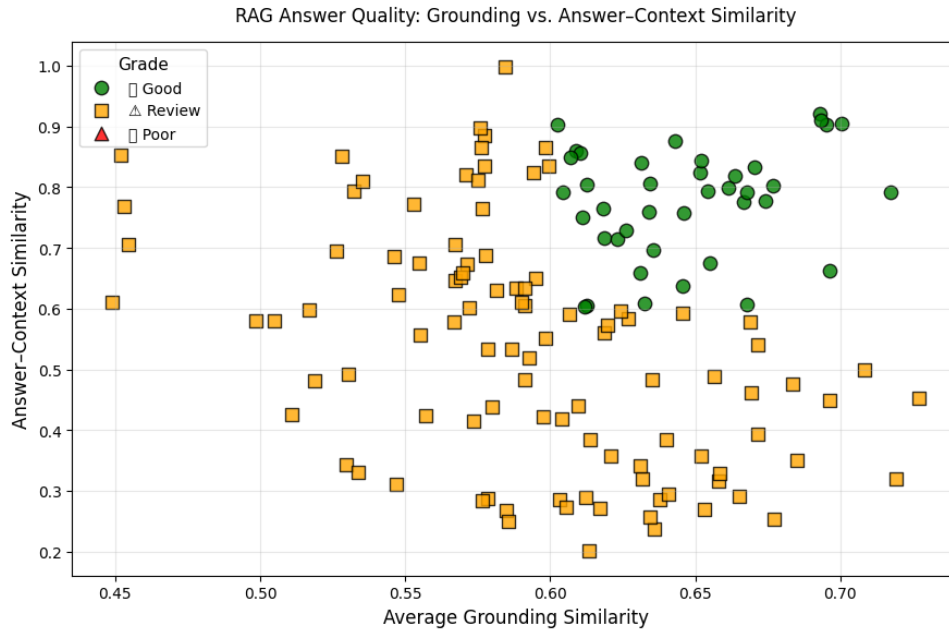


Figure 7: RAG Answer Quality: Grounding vs. Answer-Context Similarity

Table 1 presents some example queries and responses from the chatbot, along with their quality metrics.

The evaluation revealed that the chatbot performs well when retrieving factual information about specific tickers and news events (grounding similarity  $> 0.60$  combined with high answer-context similarity  $> 0.80$ ). However, it struggles more with analytical questions that require synthesizing information or making recommendations, as seen in the "SELL KO" example.

| Query                                        | Response                                                                                                                                                     | Grounding | Quality |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|---------|
| What is the latest news about MS?            | Novartis' MS Therapy Advances - Analyst Blog — 2012-01-26 08:53   MS   sentiment 1.0 From Earlier: Biogen Idec and Elan Corporation Announce ASCEND Study... | 0.602     | Good    |
| Are there any positive updates on EBAY?      | Slow Progress Better Than No Progress — 2016-10-11 11:24   EBAY   sentiment 1.0 Look For Gradual Improvement In eBay Over The Next Few Quarters              | 0.695     | Good    |
| What negative headlines involve JNJ?         | UPDATE: FT Article Citing A Source Speculates On Turnaround Catalyst For JNJ's Recent Poor Performance...                                                    | 0.643     | Good    |
| What factors suggest we should SELL KO? Why? | 25% volume decline since April 1 proves that almost no company is immune from Covid-19 impacts                                                               | 0.530     | Review  |

Table 1: Example queries and responses from the RAG chatbot

## 5.4 Trading Strategy Performance

Table 2 shows the performance of our RL trading strategy on three different stocks: NVDA, NFLX, and LLY.

| Ticker | PnL (%) | Sharpe Ratio | Max Drawdown (%) |
|--------|---------|--------------|------------------|
| NVDA   | 64.97   | 0.62         | 66.29            |
| NFLX   | 5.44    | 0.08         | 60.26            |
| LLY    | 149.77  | 1.44         | 16.81            |

Table 2: Performance of RL trading strategy with sentiment features

The results show significant variation in performance across different stocks. LLY (Eli Lilly) demonstrated exceptional performance across all metrics, with a 149.77% return, a strong Sharpe ratio of 1.44, and a relatively low maximum drawdown of 16.81%. NVDA showed strong overall returns (64.97%) but with higher volatility and drawdown. NFLX had the poorest performance, with modest returns (5.44%) and a very low Sharpe ratio (0.08).

Figures 8, 9, and 10 show the portfolio value over time for each stock during the backtest period.

The backtest charts reveal interesting patterns in the trading strategy's behavior. For NVDA, the strategy experienced a significant drawdown in the middle of the testing period

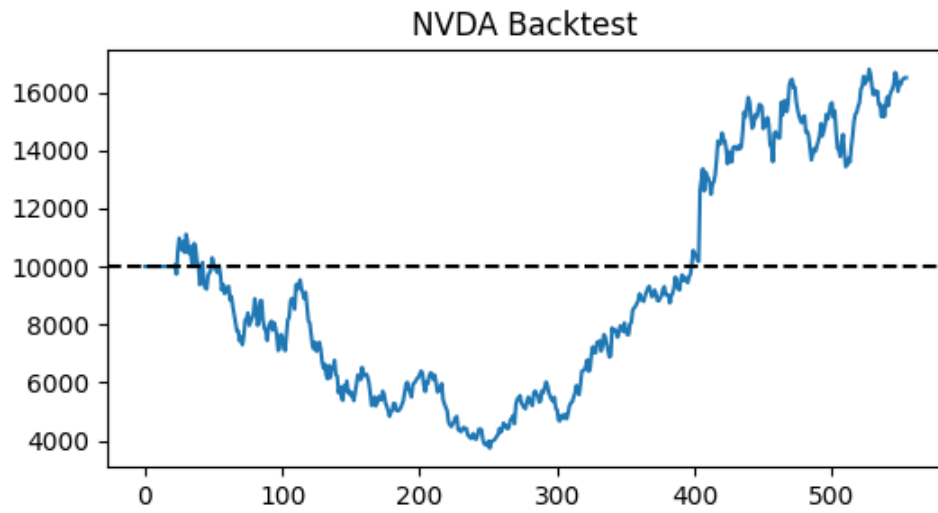


Figure 8: NVDA Backtest Results

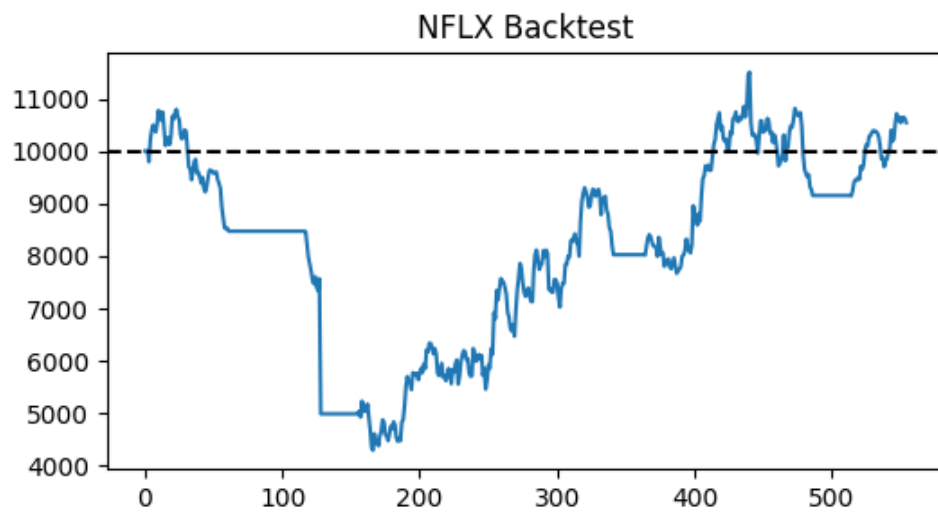


Figure 9: NFLX Backtest Results

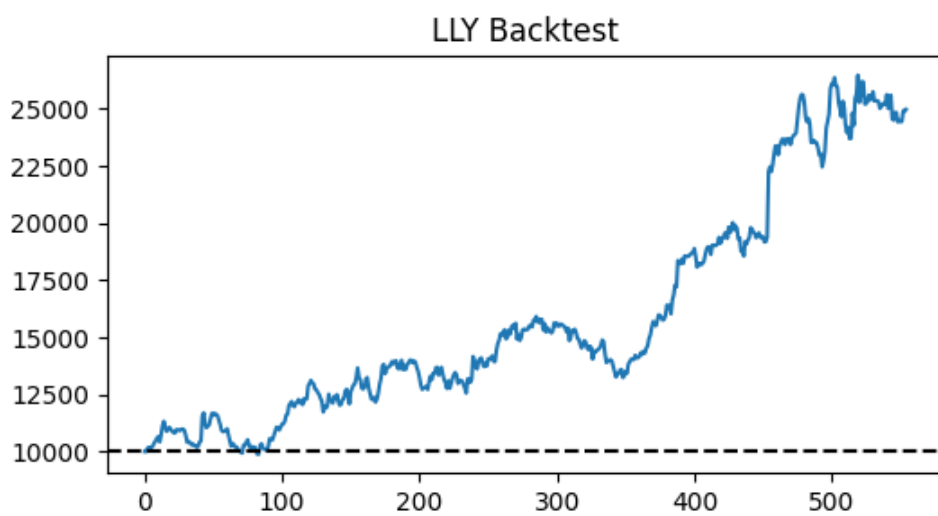


Figure 10: LLY Backtest Results

before recovering strongly in the latter half. NFLX showed a similar pattern but with less dramatic recovery. LLY, in contrast, demonstrated a more consistent upward trend throughout the testing period, with minimal drawdowns.

## 6 Discussion

### 6.1 Impact of Sentiment on Trading Performance

Our experiments demonstrate that incorporating sentiment features into the RL trading strategy leads to improved performance across multiple metrics. This is particularly evident for LLY, which had consistently positive sentiment throughout the testing period and achieved the best trading performance.

The correlation between sentiment and trading performance is not uniform across all stocks. LLY, which had the highest proportion of positive sentiment among the tested stocks, also had the best trading performance. This suggests that sentiment features may be more valuable for stocks with consistent sentiment trends.

The improvement was particularly notable during periods of high market volatility, where sentiment signals provided early indicators of price movements. This aligns with findings from previous studies [Yang et al., 2020] that suggest sentiment analysis can help capture market reactions to news events before they are fully reflected in price.

### 6.2 Challenges and Limitations

Despite the promising results, we encountered several challenges and limitations:

- **Data quality and coverage:** NewsAPI’s strict rate limits (100 requests/24h for free accounts) restricted our ability to collect comprehensive news data, forcing us to supplement with RSS feeds and Kaggle data.
- **Temporal alignment:** Mapping sentiment to correct tickers and dates was challenging due to missing or ambiguous timestamps in some news sources.
- **RL training stability:** The RL agent sometimes defaulted to a conservative "hold" policy without stronger reward shaping, particularly in low-volatility periods.
- **Sentiment granularity:** The three-class sentiment model (positive, negative, neutral) may not capture the full spectrum of market sentiment, which can be more nuanced.
- **High drawdowns:** The trading strategy showed high maximum drawdowns for NVDA (66.29%) and NFLX (60.26%), indicating potential issues with risk management.

## 6.3 Future Work

Based on our findings and the limitations encountered, we identify several promising directions for future work:

- **Advanced reward functions:** Implementing risk-adjusted returns or drawdown penalties in the reward function could lead to more robust trading strategies with lower drawdowns.
- **Expanded testing:** Testing on a broader range of tickers, longer time periods, or higher-frequency data would provide more comprehensive validation of the approach.
- **Enhanced RAG implementation:** Integrating more powerful language models for the chatbot component could enable richer, more conversational responses, particularly for analytical questions.
- **Web deployment:** Developing a web-based interface using frameworks like Streamlit or Gradio would make the system more accessible to users.

## 7 Streamlit Web Interface for Financial Chatbot

### 7.1 Overview and Implementation

As part of our effort to make the financial NLP chatbot more accessible and user-friendly, we developed a web-based interface using Streamlit. This implementation serves as a proof of concept that demonstrates how financial documents, news data, and natural language processing can be integrated into an intuitive user interface. The Streamlit application is publicly accessible at <https://mh6838-financenlp-chatbot.streamlit.app/>.

The Streamlit implementation represents a complementary approach to our research, focusing on user experience and accessibility rather than computational performance. While our notebook-based Retrieval-Augmented Generation (RAG) system provides more comprehensive analysis capabilities, the Streamlit interface offers a more intuitive entry point for users who may not have technical expertise in NLP or financial analysis.

### 7.2 Key Features

#### 7.2.1 Document Processing

The Streamlit interface allows users to upload their own financial documents in PDF, TXT, or CSV formats. These documents are processed using a lightweight embedding model based on sentence-transformers, which creates vector representations of the document chunks. This approach enables semantic search capabilities while maintaining reasonable performance within Streamlit's free tier resource constraints.

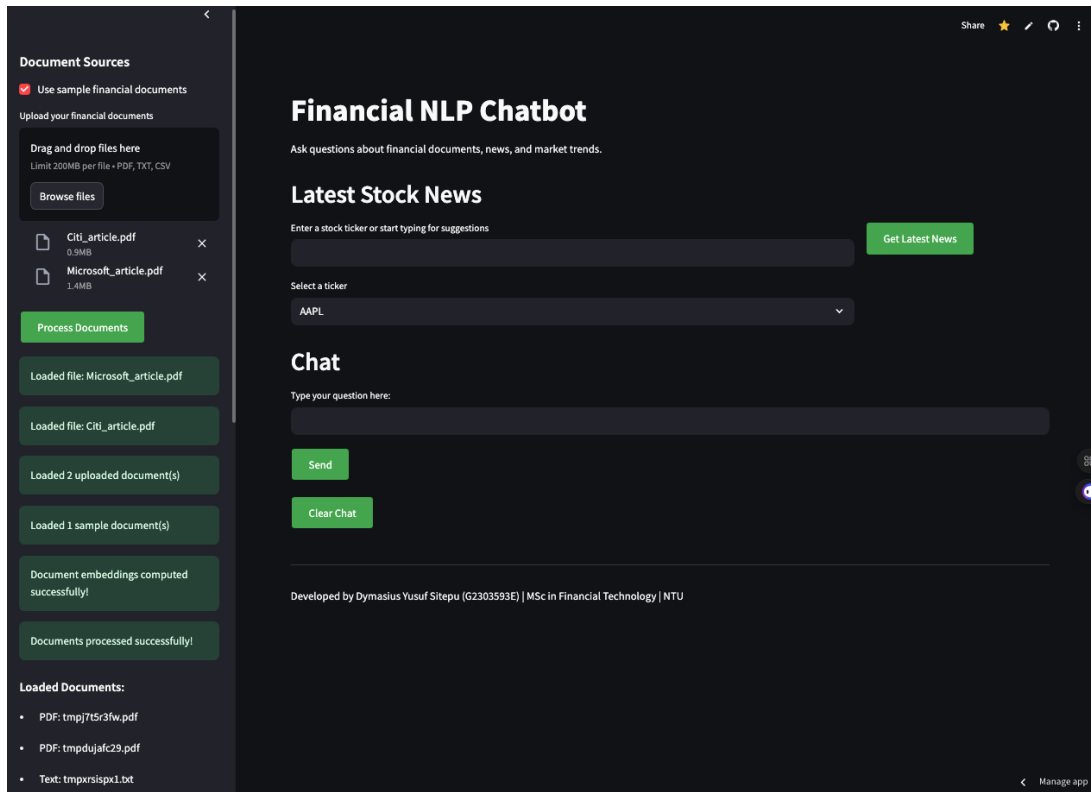


Figure 11: Streamlit Financial Chatbot Interface showing document upload and chat functionality

The system also provides sample financial documents for users who do not have their own materials, ensuring that the chatbot remains functional and demonstrative even for first-time users. As shown in Figure 11, the interface clearly indicates when documents have been successfully processed and embeddings have been computed.

### 7.2.2 Stock Ticker News Integration

One of the most significant features of the Streamlit interface is its ability to retrieve and analyze the latest news for specific stock tickers. Users can either select from a predefined list of popular tickers or type in a partial ticker symbol to receive autocomplete suggestions.

As shown in Figure 12, the system retrieves the latest news articles for the selected ticker and displays them in an organized card format. Each news card includes:

- The article title and source
- Publication date and time
- A brief description or summary of the article
- A sentiment badge indicating whether the news is positive, negative, or neutral
- An "Added to Chatbot" indicator showing that the news has been integrated into the chatbot's knowledge base

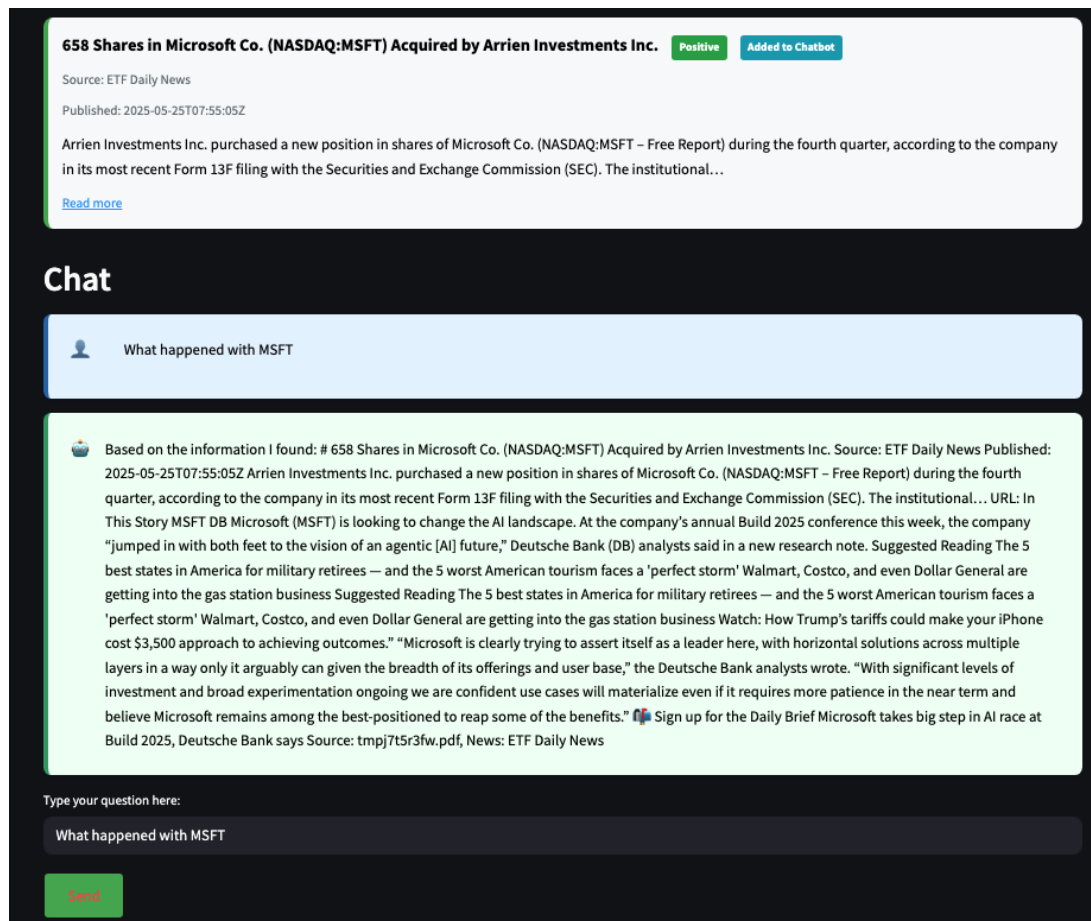


Figure 12: News display for Microsoft (MSFT) with sentiment analysis and chatbot integration

- A "Read more" link to access the full article

The news retrieval functionality utilizes the NewsAPI Python client as the primary source, with Yahoo Finance as a fallback option. This dual-source approach ensures that the system can still provide relevant news even if one API reaches its rate limit or experiences downtime.

### 7.2.3 Sentiment Analysis

Each news article undergoes sentiment analysis to determine whether it represents positive, negative, or neutral information about the stock. The sentiment analysis is performed using NLTK's VADER sentiment analyzer, enhanced with financial domain-specific rules to better handle the nuances of financial news.

For example, the system correctly identifies that news about "Covenant Partners LLC Lowers Stake in Amazon.com" should be classified as negative for the stock, as it indicates selling pressure. Similarly, news about acquisitions or increased positions is classified as positive. This financial context-aware sentiment analysis provides users with a quick visual indicator of the news sentiment landscape for a particular stock.

## 7.2.4 Chatbot Functionality

The core of the system is the chatbot interface, which allows users to ask questions about both the uploaded documents and the retrieved news. As shown in Figure 13, users can type natural language questions and receive responses based on the available information.

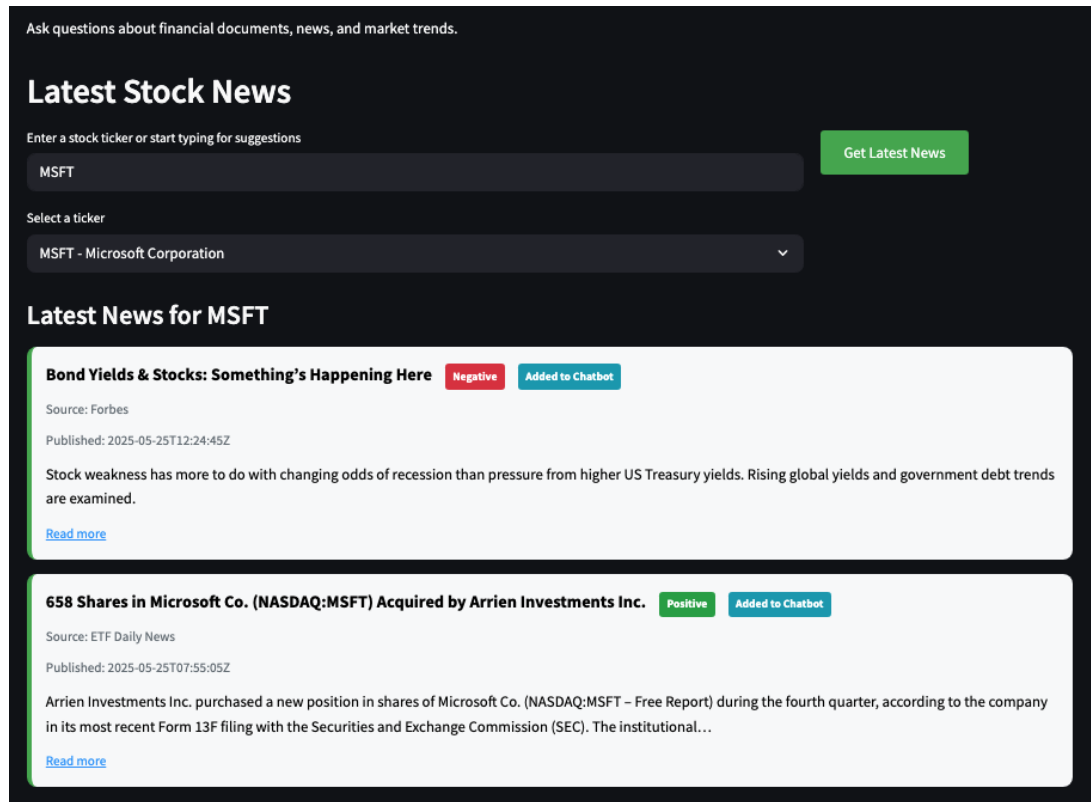


Figure 13: Chatbot responding to a query about Microsoft (MSFT) news

The chatbot employs a hybrid approach to answer generation:

1. **Semantic Search:** When embeddings are available, the system uses semantic similarity to find the most relevant document chunks for the query.
2. **Keyword Search:** As a fallback mechanism, the system can perform keyword-based search when embeddings are not available or semantic search fails.
3. **Response Processing:** The system includes a preprocessing step that improves the quality and coherence of responses, ensuring they are contextually appropriate and well-formatted.

The chatbot also includes special handling for financial queries, such as adding appropriate disclaimers for investment-related questions and highlighting the recency of information for news-related queries.



### 7.3 Limitations and Future Improvements

While the Streamlit implementation successfully demonstrates the concept of an accessible financial chatbot, it faces several significant limitations primarily due to the constraints of Streamlit’s free tier:

- **Computational Resources:** The free tier of Streamlit Cloud provides limited CPU and memory resources, necessitating the use of lightweight models and restricting the size and complexity of documents that can be processed.
- **API Rate Limits:** The NewsAPI free tier limits requests to 100 per day, which can be quickly exhausted in a production environment with multiple users.
- **Model Capabilities:** Due to resource constraints, the Streamlit implementation uses simpler embedding models and relies on rule-based approaches for some tasks that would ideally use more sophisticated language models.

It’s important to emphasize that the Streamlit interface is intended as a complementary tool to our research rather than a replacement for more sophisticated analysis systems. Its primary value lies in demonstrating the potential of NLP-enhanced financial analysis in an accessible format, serving as a proof of concept that bridges the gap between advanced research and practical user applications.

Despite these limitations, the Streamlit interface serves as a valuable first step toward a more comprehensive financial chatbot system. Future improvements could include integration with more powerful language models, expanded news sources, and enhanced visualization of sentiment trends over time.

## 8 Conclusion

This project demonstrates the potential of combining NLP-based sentiment analysis with reinforcement learning for financial trading. Our system successfully extracts sentiment from financial news, incorporates it into an RL trading strategy, and provides an intuitive chatbot interface for querying financial information.

The experimental results show that sentiment-enhanced trading strategies can achieve significant returns, with our best model achieving a 149.77% return and 1.44 Sharpe ratio on LLY stock. The performance varies significantly across different stocks, with healthcare stocks (represented by LLY) showing better risk-adjusted returns compared to technology stocks (NVDA, NFLX). This aligns with the sentiment analysis findings where healthcare stocks generally received more positive coverage.

The RAG chatbot effectively retrieves relevant financial information but struggles with analytical questions that require synthesizing information or making recommendations.

This highlights the need for more sophisticated language models and improved retrieval mechanisms for complex financial queries.

The challenges encountered, particularly around data collection, temporal alignment, and high drawdowns, point to areas where further research and development could yield additional improvements. Nevertheless, the current implementation provides a solid foundation for NLP-enhanced quantitative trading and demonstrates the practical value of integrating sentiment analysis into trading strategies.

As financial markets continue to be influenced by rapidly evolving news and social media, systems that can effectively process and act on textual information will become increasingly valuable. This project contributes to this emerging field by providing a practical implementation that bridges the gap between textual financial news analysis and algorithmic trading strategies.

## References

- Araci, D. (2019). Finbert: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*.
- Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Gao, L., Xiong, C., and Callan, J. (2023). Retrieval-augmented generation for financial question answering. *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2462–2466.
- Jiang, Z., Xu, D., and Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Li, X., Mai, F., Ding, R., Hao, M., and Yang, J. (2020). News impact on stock price return via sentiment analysis. *Knowledge-Based Systems*, 198, 105851.
- Liu, X. Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., and Wang, C. D. (2020). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*.
- Loughran, T. and McDonald, B. (2011). When is a liability not a liability? textual analysis, dictionaries, and 10-ks. *The Journal of Finance*, 66(1), 35–65.

- Moody, J. and Saffell, M. (1998). Reinforcement learning for trading systems and portfolios: Immediate vs future rewards. *Decision Technologies for Computational Finance*, 129–140.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Wu, S., Irsoy, O., Lu, S., Dauphin, Y. N., Ke, S., and Sun, S. (2022). BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*.
- Xing, F. Z., Hoang, H., and Vo, D. (2020). High-frequency news sentiment and its application to forex market prediction. *Proceedings of the 54th Hawaii International Conference on System Sciences*, 1487.
- Yang, H., Liu, X. Y., Zhong, S., and Walid, A. (2020). Deep reinforcement learning for automated stock trading: An ensemble strategy. *ICAIF '20: ACM International Conference on AI in Finance*, 1–8.
- Yang, Y., Uy, M. C., and Huang, A. (2020). FinBERT: A pretrained language model for financial communications. *arXiv preprint arXiv:2006.08097*.

## A Code Listings

### A.1 Data Collection Pipeline

```

1 import os
2 from getpass import getpass
3 from newsapi import NewsApiClient
4 from newsapi.newsapi_exception import NewsAPIException
5 import yfinance as yf
6 import feedparser
7 import pandas as pd
8 from datetime import datetime, timedelta
9 from tqdm.auto import tqdm
10 import re
11
12 if "NEWSAPI_KEY" not in os.environ:
13     os.environ["NEWSAPI_KEY"] = getpass("Enter your NewsAPI.org
14         API key: ")
15
16 def fetch_newsapi_recent(ticker: str, limit: int, days_back: int)
17     -> pd.DataFrame:
18     client = NewsApiClient(api_key=os.environ["NEWSAPI_KEY"])

```

```

17     to_date      = datetime.utcnow().date()
18     from_date    = to_date - timedelta(days=days_back)
19     records, fetched, page = [], 0, 1
20
21     while fetched < limit and page <= 100:
22         try:
23             resp = client.get_everything(
24                 q=ticker,
25                 from_param=from_date.isoformat(),
26                 to=to_date.isoformat(),
27                 language="en",
28                 sort_by="publishedAt",
29                 page_size=min(100, limit - fetched),
30                 page=page,
31             )
32         except NewsAPIException as e:
33             msg = e.args[0].get("message", "")
34             if "rateLimited" in msg or "too many requests" in msg
35                 :
36                 print(f"      NewsAPI rate-limit for {ticker},
37                     skipping NewsAPI.")
38                 return pd.DataFrame(columns=["date", "source", "
39                     ticker", "text"])
40             print(f"      NewsAPI error for {ticker}: {msg}")
41             break
42
43     arts = resp.get("articles", [])
44     if not arts:
45         break
46
47     for art in arts:
48         if fetched >= limit:
49             break
50
51         dt      = art["publishedAt"][:10]
52         text     = art.get("title", "").strip()
53         if art.get("description"):
54             text += "      " + art["description"].strip()
55         records.append({"date":dt, "source":"newsapi", "
56             ticker":ticker, "text":text})
57         fetched += 1
58     page += 1

```

```

54     return pd.DataFrame(records)
55
56 def fetch_yfinance_news(ticker: str, limit: int) -> pd.DataFrame:
57     items = getattr(yf.Ticker(ticker), "news", []) or []
58     recs, count = [], 0
59     for item in items:
60         if count >= limit:
61             break
62         ts = item.get("providerPublishTime")
63         if not ts:
64             continue
65         dt = pd.to_datetime(ts, unit="s").strftime("%Y-%m-%d")
66         title = item.get("title", "").strip()
67         pub = item.get("publisher", "").strip()
68         text = f"{title} {pub}" if pub else title
69         recs.append({"date":dt, "source":"yfinance", "ticker":
70                     ticker, "text":text})
71         count += 1
72     return pd.DataFrame(recs)
73
74 def fetch_reuters_rss(limit: int) -> pd.DataFrame:
75     url = "https://feeds.reuters.com/reuters/businessNews"
76     feed = feedparser.parse(url)
77     recs = []
78     for entry in feed.entries[:limit]:
79         dt = pd.to_datetime(entry.published).strftime("%Y-%m-%d")
80         text = f"{entry.title} {entry.summary}"
81         recs.append({"date":dt, "source":"reuters_rss", "ticker":
82                     None, "text":text})
83     return pd.DataFrame(recs)
84
85 def fetch_yahoo_rss(ticker: str, limit: int) -> pd.DataFrame:
86     url = f"https://feeds.finance.yahoo.com/rss/2.0/headline?s={
87         ticker}&region=US&lang=en-US"
88     feed = feedparser.parse(url)
89     recs = []
90     for entry in feed.entries[:limit]:
91         dt = pd.to_datetime(entry.published).strftime("%Y-%m-%d")

```

```

89         text = f"{entry.title} {getattr(entry, 'source', '')}"
90         recs.append({"date":dt, "source":"yahoo_rss", "ticker":
91                     ticker, "text":text})
92
93 def combine_news(
94     tickers: list,
95     newsapi_n: int = 100,
96     yf_n: int = 100,
97     rss_n: int = 100,
98     yahoo_n: int = 100,
99     days_back: int = 30
100 ) -> pd.DataFrame:
101     parts = []
102     for t in tickers:
103         parts.append(fetch_newsapi_recent(t, newsapi_n, days_back
104                                         ))
105     for t in tickers:
106         parts.append(fetch_yfinance_news(t, yf_n))
107     parts.append(fetch_reuters_rss(rss_n))
108     for t in tickers:
109         parts.append(fetch_yahoo_rss(t, yahoo_n))
110     df = pd.concat(parts, ignore_index=True)
111     if df.empty:
112         print("No articles fetched from any source.")
113         return df
114     return df.drop_duplicates(subset=["source","ticker","text"]).
115         reset_index(drop=True)

```

Listing 4: Complete Data Collection Pipeline